

P0020r5 : Floating Point Atomic

Project: ISO JTC1/SC22/WG21: Programming Language C++
Number: P0020r5
Date: 2017-03-06
Reply-to: hcedwar@sandia.gov
Author: H. Carter Edwards
Contact: hcedwar@sandia.gov
Author: Hans Boehm
Contact: hboehm@google.com
Author: Olivier Giroux
Contact: ogiroux@nvidia.com
Author: JF Bastien
Contact: jfbastien@apple.com
Author: James Reus
Contact: reus1@llnl.gov
Audience: Library Evolution
URL: <https://github.com/kokkos/ISO-CPP-Papers/blob/master/P0020.rst>

Revision History

P0020r3

- Align proposal with content of corresponding sections in N5131, 2016-07-15.

P0020r4

- Editorial, add hyphenation to "floating point"
- 2016-11-09 Issaquah SG1 decision: move to LEWG targeting C++20

P0020r5

- 2017-03-01 Kona LEWG review and consensus: move to C++20 IS
- Change remarks regarding floating point environment from note to normative.
- Query vendors regarding changing "should" to "shall" in regard to floating point environment. Response from two vendors supporting atomic floating point add operations in hardware was "do not change."
- Align with C++17 working draft atomic wording as per P0558r1
- Note: Section and paragraph numbers have not yet been updated to C++17 working draft new section numbering.

Overview / Motivation

This paper proposes an extension to the atomic operations library [atomics] for atomic addition on an object conforming to the `atomic<T>` where T is a *floating-point* type (N5131 3.9.1p8).

The capability for atomic addition on floating-point types critical for parallel high performance computing (HPC) applications.

Proposal:

add section to 32.7 Specializations for floating-point types

There shall be specializations of the **atomic** template for floating-point types `float`, `double`, `long double`. For each such floating-point type *floating-point*, the specialization `atomic<floating-point>` provides additional atomic operations appropriate to floating-point types.

```

template<> struct atomic< floating-point > {
    static constexpr bool is_always_lock_free = implementation-defined ;
    bool is_lock_free() const volatile noexcept;
    bool is_lock_free() const noexcept;
    void store(floating-point , memory_order = memory_order_seq_cst ) volatile noexcept;
    void store(floating-point , memory_order = memory_order_seq_cst ) noexcept;
    floating-point load( memory_order = memory_order_seq_cst ) volatile noexcept;
    floating-point load( memory_order = memory_order_seq_cst ) noexcept;
    operator floating-point () volatile noexcept ;
    operator floating-point () noexcept ;
    floating-point exchange(floating-point , memory_order = memory_order_seq_cst ) volatile noexcept;
    floating-point exchange(floating-point , memory_order = memory_order_seq_cst ) noexcept;
    bool compare_exchange_weak( floating-point & , floating-point , memory_order , memory_order ) volatile noexcept;
    bool compare_exchange_weak( floating-point & , floating-point , memory_order , memory_order ) noexcept;
    bool compare_exchange_strong( floating-point & , floating-point , memory_order , memory_order ) volatile
    noexcept;
    bool compare_exchange_strong( floating-point & , floating-point , memory_order , memory_order ) noexcept;
    bool compare_exchange_weak( floating-point & , floating-point , memory_order = memory_order_seq_cst ) volatile
    noexcept;
    bool compare_exchange_weak( floating-point & , floating-point , memory_order = memory_order_seq_cst )
    noexcept;
    bool compare_exchange_strong( floating-point & , floating-point , memory_order = memory_order_seq_cst ) volatile
    noexcept;
    bool compare_exchange_strong( floating-point & , floating-point , memory_order = memory_order_seq_cst )
    noexcept;

    floating-point fetch_add( floating-point , memory_order = memory_order_seq_cst) volatile noexcept;
    floating-point fetch_add( floating-point , memory_order = memory_order_seq_cst) noexcept;
    floating-point fetch_sub( floating-point , memory_order = memory_order_seq_cst) volatile noexcept;
    floating-point fetch_sub( floating-point , memory_order = memory_order_seq_cst) noexcept;

    atomic() noexcept = default ;
    constexpr atomic( floating-point ) noexcept ;
    atomic( const atomic & ) = delete ;
    atomic & operator = ( const atomic & ) = delete ;
    atomic & operator = ( const atomic & ) volatile = delete ;
    floating-point operator=( floating-point ) volatile noexcept ;
    floating-point operator=( floating-point ) noexcept ;

    floating-point operator+=( floating-point ) volatile noexcept;
    floating-point operator+=( floating-point ) noexcept;
    floating-point operator-=( floating-point ) volatile noexcept;
    floating-point operator-=( floating-point ) noexcept;
};

```

The atomic floating-point specializations shall be standard-layout structs. They shall each have a trivial default constructor and a trivial destructor.

Descriptions are provided below only for members that differ from the primary template.

T A::fetch_key(T operand, memory_order order = memory_order_seq_cst) volatile noexcept;
T A::fetch_key(T operand, memory_order order = memory_order_seq_cst) noexcept;

Effects: Atomically replaces the value pointed to by `this` with the result of the computation applied to the value pointed to by `this` and the given operand. Memory is affected according to the value of `order`. These operations are atomic read-modify-write operations (4.7).

Returns: Atomically, the value pointed to by `this` immediately before the effects.

Remark: If the result is not a representable value for its type (5p4) the result is unspecified, but the operations otherwise have no undefined behavior. Atomic arithmetic operations on *floating-point* should conform to `std::numeric_limits< floating-point >` traits associated with the floating-point type (18.3.2). The floating-point environment (26.4) for atomic arithmetic operations on *floating-point* may be different than the calling thread's

floating-point environment.